

PROJECT DEMONSTRATION PHASE

AI FAQ Assistant API

1. Purpose of Demonstration

This document walks through the working demonstration of the AI FAQ Assistant API, showing that all core modules — authentication, FAQ management, keyword search, and AI-assisted content generation — function correctly end to end. Each endpoint was tested using Postman inside Visual Studio Code, with the live server running locally on port 5000.

2. Environment Setup

2.1 Install Dependencies

```
npm install
```

2.2 Configure Environment Variables (.env)

```
PORT=5000 MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant  
JWT_SECRET=your_jwt_secret_key_here GEMINI_API_KEY=your_google_gemini_api_key_here
```

2.3 Start the Server

Development mode (auto-reload on file changes):

```
npm run dev
```

Production mode:

```
npm start
```

On a successful start, the terminal confirms the database connection and prints: "Server running in development mode on port 5000".

3. Testing Methodology

A pre-built Postman collection (AI_FAQ_Assistant_API.postman_collection.json) is included with the project. It defines two collection variables — baseUrl (http://localhost:5000) and token — and includes a test script on the Login request that automatically captures the returned JWT into {{token}}, so every subsequent protected request is authorised automatically.

- Public endpoints (register, login, list/search FAQs) were tested with no Authorization header.
- Protected endpoints (create/update/delete FAQ, AI routes) were tested with an Authorization: Bearer <token> header.

4. Step-by-Step Demonstration

Step 1 — User Registration

- **Endpoint:** POST /api/auth/register
- **Access:** Public

A new user is registered with a name, email, and password. The server hashes the password with bcrypt, stores the user, and immediately returns a signed JWT token.

The screenshot displays a Postman interface for a POST request to `http://localhost:5000/api/auth/register`. The request body is a JSON object: `{ "name": "Fathima", "email": "fathima22@gmail.com", "password": "fathu@22" }`. The response is a `201 Created` status with a `328 Bytes` size and `427 ms` time. The response body is a JSON object: `{ "success": true, "message": "User registered successfully", "data": { "_id": "6ab3b3dd2232e0119de9687d", "name": "Fathima", "email": "fathima22@gmail.com", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhYjNiM2RkMjIzMmUwMTESZGU5Njg3ZCIsIm1hdCI6MTc5MDE2MTg4NSwiZXhwIjoxNzkyNzUzODg1fQ.Radf6tWk2-N-F1NpCWdujKcQn8RDIsu7-4dU-HwVrFQ" } }`.

```
POST http://localhost:5000/api/auth/register
```

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "name": "Fathima",
3   "email": "fathima22@gmail.com",
4   "password": "fathu@22"
5 }
```

Status: 201 Created Size: 328 Bytes Time: 427 ms

Response Headers 7 Cookies Results Docs

```
1 {
2   "success": true,
3   "message": "User registered successfully",
4   "data": {
5     "_id": "6ab3b3dd2232e0119de9687d",
6     "name": "Fathima",
7     "email": "fathima22@gmail.com",
8     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
          .eyJpZCI6IjZhYjNiM2RkMjIzMmUwMTESZGU5Njg3ZCIsIm1
          hdCI6MTc5MDE2MTg4NSwiZXhwIjoxNzkyNzUzODg1fQ
          .Radf6tWk2-N-F1NpCWdujKcQn8RDIsu7-4dU-HwVrFQ"
9   }
10 }
```

Step 2 — User Login

- **Endpoint:** POST /api/auth/login
- **Access:** Public

The registered user logs in with the same email and password. The server verifies the bcrypt hash and returns a fresh JWT token used to authorise later requests.

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: `http://localhost:5000/api/auth/login`
 - Body (JSON):

```
1 {
2   "email": "fathima22@gmail.com",
3   "password": "fathu@22"
4 }
5
```
- Response:**
 - Status: 200 OK
 - Size: 316 Bytes
 - Time: 171 ms
 - Body (JSON):

```
1 {
2   "success": true,
3   "message": "Login successful",
4   "data": {
5     "_id": "6ab3b3dd2232e0119de9687d",
6     "name": "Fathima",
7     "email": "fathima22@gmail.com",
8     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhYjNiM2RkMjIzZWUwMTESZGU5Njg3ZCI6Im1hdCI6MTc5MDE2MjAwNiwiZXhwIjoxNzkyNzU0MDA2fQ.ZRiF4DzHKWZ5uDgFDPzBPiR2aset3hdUmwUKOGpPpQ4"
9   }
10 }
```

Step 3 — Create a FAQ (Protected)

- **Endpoint:** POST /api/faqs
- **Access:** Private — requires Bearer <token> from Step 2

An authenticated user submits a question, answer, and category. The FAQ is validated against the schema (category must be one of the allowed enum values) and saved with a createdBy reference to the logged-in user.

The screenshot displays a REST client interface with a POST request to `http://localhost:5000/api/faqs`. The request body is a JSON object with the following content:

```
1 {
2   "question": "How to Reset my password",
3   "answer": "Contact IT team",
4   "category": "Technology"
5 }
```

The response status is **201 Created**, with a size of **309 Bytes** and a time of **1.18 s**. The response body is a JSON object with the following content:

```
1 {
2   "success": true,
3   "message": "FAQ created successfully",
4   "data": {
5     "question": "How to Reset my password",
6     "answer": "Contact IT team",
7     "category": "Technology",
8     "createdBy": "6ab29808539c8a286fd8ef4a",
9     "_id": "6ab3b2d02232e0119de96876",
10    "createdAt": "2026-09-23T11:06:56.671Z",
11    "updatedAt": "2026-09-23T11:06:56.671Z",
12    "__v": 0
13  }
14 }
```

Step 4 — Retrieve All FAQs (Public)

- **Endpoint:** GET /api/faqs
- **Access:** Public

Any client — authenticated or not — can list published FAQs. The response includes the populated creator's name and email alongside each FAQ.

The screenshot displays a REST client interface with a GET request to `http://localhost:5000/api/faqs` and its corresponding JSON response. The response status is 200 OK, with a size of 679 Bytes and a time of 282 ms.

Request:

```
GET http://localhost:5000/api/faqs
```

Response:

```
{
  "success": true,
  "message": "FAQs retrieved successfully",
  "count": 2,
  "data": [
    {
      "_id": "6ab3b2d02232e0119de96876",
      "question": "How to Reset my password",
      "answer": "Contact IT team",
      "category": "Technology",
      "createdBy": {
        "_id": "6ab29808539c8a286fd8ef4a",
        "name": "Priya",
        "email": "priya@writeflow.com"
      },
      "createdAt": "2026-09-23T11:06:56.671Z",
      "updatedAt": "2026-09-23T11:06:56.671Z",
      "__v": 0
    },
    {
      "_id": "6ab377f2bf6c940a3557b490",
      "question": "How to Reset my password",
      "answer": "Contact IT team",
      "category": "Technology",
      "createdBy": {
        "_id": "6ab29808539c8a286fd8ef4a"
      }
    }
  ]
}
```

5. Additional Endpoints Verified

The following endpoints were also exercised via the Postman collection during the demonstration, confirming correct request/response behaviour:

Endpoint	Method	Access	Purpose
/api/auth/profile	GET	Private	Returns the logged-in user's profile details.
/api/faqs/:id	GET	Public	Fetches a single FAQ by its ID.
/api/faqs/:id	PUT	Private (owner)	Updates a FAQ's question/answer/category.
/api/faqs/:id	DELETE	Private (owner)	Deletes a FAQ owned by the requester.
/api/faqs/search?q=	GET	Public	Keyword search across question, answer, and category.
/api/ai/answer	POST	Private	Returns a direct AI-generated answer via Gemini (gemini-2.0-flash).
/api/ai/generate-faq	POST	Private	Generates a structured {question, answer} pair via Gemini (gemini-2.5-flash) for a given topic.

6. Results & Observations

- All authentication flows (register, login, protected profile access) returned the expected status codes and JWT-secured responses.
- FAQ CRUD operations correctly enforced ownership, schema validation rejected invalid data, and keyword search returned relevant matches.
- AI endpoints (backed by Google Gemini) returned well-formed answers and structured FAQ pairs, verifying the external AI integration works as designed.
- Centralised error handling returned consistent, sanitised JSON error responses for invalid or unauthorised requests.

7. Reference Links

API Testing Video:

https://drive.google.com/file/d/1u_w1sigiOynhmqTDbj4NBm3CLIMunEWN/view?usp=sharing

Full Demo Video:

https://drive.google.com/file/d/1ajAZGgOaowPdyW2x5otcsLI84NeomSYJ/view?usp=drive_link

Source Code Drive: https://drive.google.com/drive/folders/1aUqV5q25fE_iUPwWcLt02eJ-v-ytxpjz?usp=sharing

8. Conclusion

The demonstration confirms that the AI FAQ Assistant API is fully functional: secure authentication, complete FAQ CRUD operations, public search, and Gemini-powered AI content generation all behave as documented. The Postman collection provides a repeatable test suite that can be re-run at any time to validate the API before deployment or further development.